

Context-Aware Multi-Provider AI Architecture

Ultra-Minimalistik "Rise" Ilovasi uchun Murakkab AI va Shaxsiy Ma'lumotlar Integratsiyasi

1. Tizim Arxitekturası va Ishlash Prinsipi

Foydalanuvchining tizimdagi barcha ma'lumotlariga (Challenges, Daily Logs, Notes) xavfsiz va aqlli kirish imkoniyatini yaratish uchun **Context Injection (Kontekstni kiritish)** modelidan foydalanamiz. Bu usul an'anaviy chatlardan farqli o'laroq, foydalanuvchi savol berganida ma'lumotlar bazasidan tegishli ma'lumotlarni yig'ib, AI modeliga tayyor kontekst sifatida taqdim etadi.

- Auth & Security:** NextAuth.js orqali JWT asosidagi xavfsiz login/parol tizimi. Har bir AI so'rovi faqat tegishli foydalanuvchining `userId` siga tegishli ma'lumotlarni ko'ra oladi.
- Dynamic Provider Routing:** Foydalanuvchi yoki tizim ma'muri istalgan vaqtda AI provayderini (OpenAI, Anthropic, Gemini, DeepSeek) o'zgartirishi mumkin. API kalitlari va tanlangan provayder ma'lumotlar bazasida xavfsiz saqlanadi.

2. Ma'lumotlar Bazasi Sxemasi (Prisma ORM)

AI foydalanuvchining barcha ma'lumotlarini o'qishi uchun munosabatlar (relations) to'g'ri shakllantirilgan sxema kerak. Quyida authentication, challenge'lar va AI sozlamalarini qamrab olgan ultra-minimal sxema keltirilgan:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  passwordHash String
  name        String?
  createdAt   DateTime     @default(now())
  challenges  Challenge[]
  aiSettings  AiSetting?
}

model Challenge {
  id          String      @id @default(cuid())
  userId      String
  title       String
  description String?
  category    String
  durationDays Int
```

```

    createdAt    DateTime    @default(now())
    user          User        @relation(fields: [userId], references: [id], onDelete: Cascade)
    dailyLogs     DailyLog[]
  }

  model DailyLog {
    id            String      @id @default(cuid())
    challengeId   String
    dayNumber     Int
    isCompleted   Boolean     @default(false)
    note          String?
    createdAt     DateTime    @default(now())
    challenge     Challenge   @relation(fields: [challengeId], references: [id], onDelete: Cascade)
  }

  model AiSetting {
    id            String      @id @default(cuid())
    userId        String      @unique
    provider      String      @default("openai") // openai, anthropic, gemini, deepseek
    apiKey        String?     // Shifrlangan holda saqlanishi tavsiya etiladi
    customModel   String?     // Masalan: gpt-4o, claude-3-5-sonnet, deepseek-chat
    user          User        @relation(fields: [userId], references: [id], onDelete: Cascade)
  }

```

3. Universallashtirilgan AI Proвайder Adapteri

Backend qismida barcha provayderlarni bitta umumiy interfeysga keltirish uchun `/app/api/ai/route.ts` marshrutida quyidagi murakkab logikadan foydalanamiz. Bu kod kontekstni yig'adi va tanlangan API'ga yuboradi:

```

import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
import { getServerSession } from 'next-auth';

export async function POST(req: Request) {
  const session = await getServerSession();
  if (!session?.user) return new NextResponse('Unauthorized', { status: 401 });

  const { message } = await req.json();
  const userId = session.user.id;

  // 1. Foydalanuvchining barcha ma'lumotlarini (Kontekst) yig'ish
  const userData = await prisma.user.findUnique({
    where: { id: userId },
    include: {
      challenges: {
        include: { dailyLogs: true }
      },
      aiSettings: true
    }
  });

  if (!userData) return new NextResponse('User not found', { status: 404 });

```

```

// 2. Ma'lumotlarni matnli kontekst holatiga keltirish
const contextString = userData.challenges.map(c => {
  const completedDays = c.dailyLogs.filter(l => l.isCompleted).length;
  const logsNotes = c.dailyLogs.filter(l => l.note).map(l => `Kun ${l.dayNumber}: ${l.note}`)
    .join('; ');
  return `Challenge: "${c.title}" (${c.category}). Progress: ${completedDays}/${c.durationDays} kun bajarildi. Qaydlar: [${logsNotes}]`;
}).join('\n');

const systemPrompt = `Siz foydalanuvchining shaxsiy rivojlanish bo'yicha sun'iy intellekt yordamchisiz. Foydalanuvchining ilovadagi barcha ma'lumotlari quyida keltirilgan. Ushbu ma'lumotlarga tayanib, unga aniq, minimalistik va motivatsion javoblar bering:\n\nKONTEKST:\n${contextString}`;

const provider = userData.aiSettings?.provider || 'openai';
const apiKey = userData.aiSettings?.apiKey || process.env.GLOBAL_AI_API_KEY;
const model = userData.aiSettings?.customModel || 'gpt-4o';

// 3. Dinamik Provayder Routeri
try {
  let aiResponse = "";

  if (provider === 'openai' || provider === 'deepseek') {
    const baseUrl = provider === 'deepseek' ? 'https://api.deepseek.com' : 'https://api.openai.com/v1';
    const res = await fetch(`${baseUrl}/v1/chat/completions`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${apiKey}` },
      body: JSON.stringify({
        model: model,
        messages: [
          { role: 'system', content: systemPrompt },
          { role: 'user', content: message }
        ]
      })
    });
    const data = await res.json();
    aiResponse = data.choices[0].message.content;
  }
  // Bu yerga Anthropic va Gemini uchun qo'shimcha 'else if' shartlari yoziladi...

  return NextResponse.json({ response: aiResponse });
} catch (error) {
  return new NextResponse('AI Integration Error', { status: 500 });
}
}

```

4. Ultra-Minimalistik UI/UX Dizayn Qoidalari

Ko'zni charchatmaydigan, Apple va Vercel uslubidagi minimalistik dizaynga erishish uchun frontend komponentlarida quyidagi Tailwind CSS qoidalariga qat'iy amal qilinishi kerak:

Element	Eski Yondashuv (Qo'pol)	Minimalistik Yondashuv (Premium)
Kartalar va Chegaralar	Katta soyalar (<code>shadow-lg</code>), oq fon	Soyalarsiz, o'ta nozik chegara: <code>border border-zinc-200</code> yoki to'q rejimda <code>border-zinc-800</code>
Ranglar palitrasi	Neon, yorqin binafsha va yashil	Monoxrom (oq, kulrang, qora). Faqatgina muhim indikatorlar uchun juda yumshoq rang urg'ulari.
Kiritish maydonlari	To'liq ramkali og'ir <code>textarea</code>	Ramkasiz dizayn, yengil fon: <code>bg-zinc-50 dark:bg-zinc-900 focus:ring-1</code>